

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ
И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ
ТЕХНОЛОГИЙ**

**Отчет по лабораторной работе №6
по курсу «Проектирование и реализация баз данных»
Тема: «Работа с БД в СУБД MongoDB»**

Выполнила:
Блинова Полина Вячеславовна K3239

Проверила:
Говорова Марина Михайловна

Санкт-Петербург
2026 г.

Оглавление

1 Цель работы	3
2 Практическое задание (вариант 2.2 – авторские триггеры)	4
3 Выполнение работы ЛР 6.01	8
Задание 1. Установка MongoDB	8
Задание 2. Проверка работоспособности системы запуском клиента mongo	9
Задание 3а. Выполнение метода db.help()	9
Задание 3б. Выполнение метода db.help	11
Задание 3с. Выполнение метода db.stats()	12
Задание 4. Создание БД learn	12
Задание 5. Получение списка доступных БД	13
Задание 6. Создание коллекции unicorns и вставка документа	13
Задание 7. Просмотр списка текущих коллекций	13
Задание 8. Переименование коллекции unicorns	14
Задание 9. Просмотр статистики коллекции	14
Задание 10. Удаление коллекции	15
Задание 11. Удаление БД learn	16
4 Промежуточные выводы по ЛР 6.1	17
5 Выполнение работы ЛР 6.02	18
Практическое задание 2.1.1	18
Практическое задание 2.2.1	20
Практическое задание 2.2.2	22
Практическое задание 2.2.3	22
Практическое задание 2.1.4	23
Практическое задание 2.3.1	24
Практическое задание 2.3.2	24
Практическое задание 2.3.3	25
Практическое задание 2.3.4	25
Практическое задание 3.1.1	26
Практическое задание 3.1.2	27
Практическое задание 3.2	28
Практическое задание 3.3.1	28
Практическое задание 3.3.2	29

Практическое задание 3.3.3	30
Практическое задание 3.3.4	30
Практическое задание 3.3.5	31
Практическое задание 3.3.6	32
Практическое задание 3.3.7	32
Практическое задание 3.4.1	33
Практическое задание 4.1.1	35
Практическое задание 4.2.1	37
Практическое задание 4.4.1	38
6 Промежуточные выводы по ЛР 6.2	42
7 Вывод	44

1 Цель работы

Овладеть практическими навыками установки СУБД MongoDB. Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД MongoDB 8.2.

2 Практическое задание (вариант 2.2 – авторские триггеры)

ЛР 6.01:

Установите MongoDB для обеих типов систем (32/64 бита).

Проверьте работоспособность системы запуском клиента mongo.

Выполните методы:

a) `db.help()`

b) `db.help`

c) `db.stats()`

Создайте БД learn.

Получите список доступных БД.

Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.

Просмотрите список текущих коллекций.

Переименуйте коллекцию unicorns.

Просмотрите статистику коллекции.

Удалите коллекцию.

Удалите БД learn.

ЛР 6.02:

Задание 2.1.1. Вставка документов в коллекцию

Создайте базу данных learn.

Заполните коллекцию единорогов unicorns указанными документами.

Используя второй способ, вставьте в коллекцию единорогов документ {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}.

Проверьте содержимое коллекции с помощью метода find.

Задание 2.2.1. Выборка данных

Формулировка: Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

Задание 2.2.2. Исключение полей

Формулировка: Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле _id.

Задание 2.2.3. Обратный порядок

Формулировка: Вывести список единорогов в обратном порядке добавления.

Задание 2.2.4. Оператор \$slice

Формулировка: Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

Задание 2.3.1. Операторы сравнения

Формулировка: Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

Задание 2.3.2. Оператор \$all

Формулировка: Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

Задание 2.3.3. Оператор \$exists

Формулировка: Найти всех единорогов, не имеющих ключ vampires.

Задание 2.3.4. Комбинация операторов

Формулировка: Вывести упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

Задание 3.1.1. Вложенные объекты (коллекция towns)

Формулировка: Создайте коллекцию towns с указанными документами. Сформируйте запрос для списка городов с независимыми мэрами (party="I"). Сформируйте запрос для списка беспартийных мэров (party отсутствует).

Задание 3.1.2. Курсоры

Формулировка: Сформировать функцию для вывода списка самцов единорогов. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке. Вывести результат, используя `forEach`.

Задание 3.2. Агрегированные запросы

Формулировка: Вывести количество самок весом от полутонны до 600 кг. Вывести список предпочтений. Посчитать количество особей обоих полов.

Задание 3.3. Редактирование данных

Формулировка: Выполнить последовательность операций обновления данных.

Задание 3.4. Удаление данных из коллекции

Формулировка: Создайте коллекцию `towns`, включающую указанные документы. Удалите документы с беспартийными мэрами. Проверьте содержание коллекции. Очистите коллекцию. Просмотрите список доступных коллекций.

Задание 4.2. Настройка индексов

Формулировка: Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

Задание 4.3. Управление индексами

Формулировка: Получите информацию о всех индексах коллекции `unicorns`. Удалите все индексы, кроме индекса для идентификатора. Попробуйте удалить индекс для идентификатора.

Задание 4.4. План запроса

Формулировка: Создайте объемную коллекцию `numbers`, задействовав курсор (100 000 документов). Выберите последних четыре документа. Проанализируйте план выполнения запроса. Создайте индекс для ключа `value`. Получите информацию о всех индексах коллекции `numbers`. Выполните запрос с индексом. Проанализируйте план выполнения запроса с

установленным индексом. Сравните время выполнения запросов с индексом и без.

3 Выполнение работы ЛР 6.01

Задание 1. Установка MongoDB

Выполнение: MongoDB установлена на macOS с помощью Homebrew.
Команды:

```
brew tap mongodb/brew
brew install mongodb-community@8.0
brew services start mongodb-community@8.0
```

Результат: MongoDB Community 8.0 успешно установлена и запущена как служба.

```
🍺 /opt/homebrew/Cellar/merve/1.2.2_1: 19 files, 151.9KB
==> Installing mongodb/brew/mongodb-community@8.0 dependency: nbytes
==> Pouring nbytes--0.1.4.arm64_tahoe.bottle.tar.gz
🍺 /opt/homebrew/Cellar/nbytes/0.1.4: 9 files, 90.4KB
==> Installing mongodb/brew/mongodb-community@8.0 dependency: simdjson
==> Pouring simdjson--4.6.4.arm64_tahoe.bottle.tar.gz
🍺 /opt/homebrew/Cellar/simdjson/4.6.4: 19 files, 8.0MB
==> Installing mongodb/brew/mongodb-community@8.0 dependency: sqlite
==> Pouring sqlite--3.53.1.arm64_tahoe.bottle.tar.gz
🍺 /opt/homebrew/Cellar/sqlite/3.53.1: 13 files, 5.3MB
==> Installing mongodb/brew/mongodb-community@8.0 dependency: uvwasi
==> Pouring uvwasi--0.0.23.arm64_tahoe.bottle.1.tar.gz
🍺 /opt/homebrew/Cellar/uvwasi/0.0.23: 15 files, 289KB
==> Installing mongodb/brew/mongodb-community@8.0 dependency: node
==> Pouring node--26.0.0.arm64_tahoe.bottle.tar.gz
🍺 /opt/homebrew/Cellar/node/26.0.0: 1,889 files, 82.4MB
==> Installing mongodb/brew/mongodb-community@8.0 dependency: mongosh
==> Pouring mongosh--2.8.3.arm64_tahoe.bottle.tar.gz
🍺 /opt/homebrew/Cellar/mongosh/2.8.3: 16,529 files, 109.0MB
==> Installing mongodb/brew/mongodb-community@8.0
==> Caveats
To start mongodb/brew/mongodb-community@8.0 now and restart at login:
  brew services start mongodb/brew/mongodb-community@8.0
==> Summary
🍺 /opt/homebrew/Cellar/mongodb-community@8.0/8.0.23: 12 files, 336.8MB, built
in 1 second
==> Running `brew cleanup mongodb-community@8.0`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
==> Caveats
==> mongodb-community@8.0
To start mongodb/brew/mongodb-community@8.0 now and restart at login:
  brew services start mongodb/brew/mongodb-community@8.0
[polinochka@Polinas-MacBook-Pro ~ % brew services start mongodb-community ]
Error: Formula `mongodb-community` is not installed.
[polinochka@Polinas-MacBook-Pro ~ % brew services start mongodb-community@8.0 ]
==> Successfully started `mongodb-community@8.0` (label: homebrew.mxcl.mongodb-c
polinochka@Polinas-MacBook-Pro ~ %
```

Задание 2. Проверка работоспособности системы запуском клиента mongo

Команда:

mongosh

Лог (результат):

```
polinoshka@Polinas-MacBook-Pro ~ % mongosh

Current Mongosh Log ID: 6a0ee0a088f1e93cf751961d
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverS
electionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.0.23
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to Mong
oDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
  2026-05-21T13:36:56.347+03:00: Access control is not enabled for the database
. Read and write access to data and configuration is unrestricted
-----

test> █
```

Вывод: Клиент MongoDB успешно подключился к серверу. Текущая база данных по умолчанию — test

Задание 3а. Выполнение метода db.help()

Команда:

db.help()

Лог (результат):

```

test> db.help()

Database Class:

getMongo                Returns the current database connection
getName                 Returns the name of the DB
getCollectionNames      Returns an array containing the names of all collections in the current database.
getCollectionInfos      Returns an array of documents with collection information, i.e. collection name and options, for the current database.
runCommand              Runs an arbitrary command on the database.
adminCommand            Runs an arbitrary command against the admin database.
aggregate              Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB            Returns another database without modifying the db variable in the shell environment.
getCollection           Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase            Removes the current database, deleting the associated data files.
createUser              Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
updateUser             Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's value
s. This includes updates to the user's roles array.
changeUserPassword     Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout                 Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser               Removes the user from the current database.
dropAllUsers            Removes all users from the current database.
auth                  Allows a user to authenticate to the database from within the shell.
grantRolesToUser        Grants additional roles to a user.
revokeRolesFromUser    Removes a one or more roles from a user on the current database.
getUser                Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method is run.
getUsers               Returns information for all the users in the database.
createCollection        Create new collection
createEncryptedCollection Creates a new collection with a list of encrypted fields each with unique and auto-created data encryption keys (DEKs). This is a utility function that internally utilizes ClientEncryption.createEncryptedCollection.
createView              Create new view
createRole              Creates a new role.
updateRole              Updates the role's profile on the database on which you run the method. An update to a field completely replaces the previous field's value
s.
dropRole                Removes the role from the current database.
dropAllRoles            Removes all roles from the current database.
grantRolesToRole        Grants additional roles to a role.
revokeRolesFromRole    Removes a one or more roles from a role on the current database.
grantPrivilegesToRole   Grants additional privileges to a role.
revokePrivilegesFromRole Removes a one or more privileges from a role on the current database.
getRole                 Returns role information for a specified role. Run this method on the role's database. The role must exist on the database on which the method is run.
getRoles                Returns information for all the roles in the database.
currentOp               Runs an aggregation using $currentOp operator. Returns a document that contains information on in-progress operations for the database instance. For further information, see $currentOp.
killOp                  Calls the killOp command. Terminates an operation as specified by the operation ID. To find operations and their corresponding IDs, see $currentOp or db.currentOp().
rrrentOp or db.currentOp().
shutdownServer          Calls the shutdown command. Shuts down the current mongod or mongos process cleanly and safely. You must issue the db.shutdownServer() operation against the admin database.
fsyncLock               Calls the fsync command. Forces the mongod to flush all pending write operations to disk and locks the entire mongod instance to prevent additional writes until the user releases the lock with a corresponding db.fsyncUnlock() command.
ditional writes until the user releases the lock with a corresponding db.fsyncUnlock() command.
fsyncUnlock             Calls the fsyncUnlock command. Reduces the lock taken by db.fsyncLock() on a mongod instance by 1.
version                 returns the db version. uses the buildinfo command
serverBits              returns the db serverBits. uses the buildInfo command
isMaster                Calls the isMaster command
hello                   Calls the hello command
serverBuildInfo          returns the db serverBuildInfo. uses the buildInfo command

currentOp               Runs an aggregation using $currentOp operator. Returns a document that contains information on in-progress operations for the database instance. For further information, see $currentOp.
killOp                  Calls the killOp command. Terminates an operation as specified by the operation ID. To find operations and their corresponding IDs, see $currentOp or db.currentOp().
shutdownServer          Calls the shutdown command. Shuts down the current mongod or mongos process cleanly and safely. You must issue the db.shutdownServer() operation against the admin database.
fsyncLock               Calls the fsync command. Forces the mongod to flush all pending write operations to disk and locks the entire mongod instance to prevent additional writes until the user releases the lock with a corresponding db.fsyncUnlock() command.
fsyncUnlock             Calls the fsyncUnlock command. Reduces the lock taken by db.fsyncLock() on a mongod instance by 1.
version                 returns the db version. uses the buildinfo command
serverBits              returns the db serverBits. uses the buildInfo command
isMaster                Calls the isMaster command
hello                   Calls the hello command
serverBuildInfo          returns the db serverBuildInfo. uses the buildInfo command

mmmand                  returns the server stats. uses the serverStatus command
serverStatus
nd
stats                  returns the db stats. uses the dbStats command
hostInfo               Calls the hostInfo command
serverCmdLineOpts       returns the db serverCmdLineOpts. uses the getCmdLineOpts command
rotateCertificates      Calls the rotateCertificates command
printCollectionStats    Prints the collection.stats for each collection in the database.
getProfilingStatus      returns the db getProfilingStatus. uses the profile command
setProfilingLevel       returns the db setProfilingLevel. uses the profile command
setLogLevel             returns the db setLogLevel. uses the setParameter command
getLogComponents        returns the db getLogComponents. uses the getParameter command
cloneDatabase            deprecated, non-functional
cloneCollection          deprecated, non-functional
copyDatabase             deprecated, non-functional
commandHelp             returns the db commandHelp. uses the passed in command with help: true
listCommands            Calls the listCommands command
getLastErrorObj          Calls the getLastError command
getLastError            Calls the getLastError command
printShardingStatus      Calls sh.status(verbose)
printSecondaryReplicationInfo Prints secondary replication information
getReplicationInfo       Returns replication information
printReplicationInfo     Formats sh.getReplicationInfo
printSlaveReplicationInfo DEPRECATED. Use db.printSecondaryReplicationInfo
setSecondaryOk           This method is deprecated. Use db.getMongo().setReadPreference() instead
watch                   Opens a change stream cursor on the database (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an experimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

test>

```

Вывод: Метод `db.help()` выводит полный список всех доступных команд для работы с базой данных.

Задание 3b. Выполнение метода db.help

Команда:

db.help

Лог (результат):

```
[test> db.help ]

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current database.
  getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
  runCommand         Runs an arbitrary command on the database.
  adminCommand       Runs an arbitrary command against the admin database.
  aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
  getSiblingDB       Returns another database without modifying the db variable in the shell environment.
  getCollection       Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
  dropDatabase        Removes the current database, deleting the associated data files.
  createUser          Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
  updateUser          Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
  logout             Ends the current authentication session. This function has no effect if the current session is not authenticated.
  dropUser            Removes the user from the current database.
  dropAllUsers        Removes all users from the current database.
  auth               Allows a user to authenticate to the database from within the shell.
  grantRolesToUser    Grants additional roles to a user.
  revokeRolesFromUser Removes a one or more roles from a user on the current database.
  getUser             Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
  getUsers            Returns information for all the users in the database.
  createCollection    Create new collection
  createEncryptedCollection Creates a new collection with a list of encrypted fields each with unique and auto-created data encryption keys (DEKs). This is a utility function that internally utilises ClientEncryption.createEncryptedCollection.
  createView          Create new view
  createRole          Creates a new role.
  updateRole          Updates the role's profile on the database on which you run the method. An update to a field completely replaces the previous field's values.
  dropRole            Removes the role from the current database.
  dropAllRoles        Removes all roles from the current database.
  grantRolesToRole    Grants additional roles to a role.
  revokeRolesFromRole Removes a one or more roles from a role on the current database.
  grantPrivilegesToRole Grants additional privileges to a role.
  revokePrivilegesFromRole Removes a one or more privileges from a role on the current database.
```

Вывод: При вводе метода db.help без круглых скобок консоль выводит не результат выполнения метода (как в случае с db.help()), а описание структуры и свойств функции help. Это особенность интерпретации JavaScript в консоли MongoDB: метод без вызова показывает свою внутреннюю реализацию и метаданные, а не выполняет действие.

Задание 3с. Выполнение метода db.stats()

Команда:

```
db.stats()
```

Лог (результат):

```
[test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test> █
```

Вывод: Статистика показывает, что база данных test пуста (0 коллекций, 0 объектов).

Задание 4. Создание БД learn

Команда:

```
use learn
```

Лог (результат):

```
[test> use learn
switched to db learn
learn> █
```

Вывод: Произошло переключение на базу данных learn. База данных будет создана автоматически при первой записи.

Задание 5. Получение списка доступных БД

Команда:

```
show dbs
```

Лог (результат):

```
[learn> show dbs
admin    40.00 KiB
config   92.00 KiB
local    40.00 KiB
_
```

Вывод: База данных learn отсутствует в списке, так как она еще пустая.

Задание 6. Создание коллекции unicorns и вставка документа

Команды:

```
db.createCollection("unicorns")
db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
```

Лог (результат):

```
[learn> db.createCollection("unicorns")
{ ok: 1 }
[learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
{
  acknowledged: true,
  insertedId: ObjectId('6a0ee33388f1e93cf751961e')
}
learn>
```

Вывод: Коллекция unicorns успешно создана. Документ вставлен, автоматически сгенерирован `_id`.

Задание 7. Просмотр списка текущих коллекций

Команда:

```
show collections
```

Лог (результат):

```
[learn> show collections
unicorns
_
```

Вывод: В базе данных learn присутствует одна коллекция — unicorns.

Задание 8. Переименование коллекции unicorns

Команда:

```
db.unicorns.renameCollection("magical_creatures")
```

Лог (результат):

```
[learn> db.unicorns.renameCollection("magical_creatures")
{ ok: 1 }
learn>
```

Проверка:

```
[learn> show collections
magical_creatures
```

Вывод: Коллекция успешно переименована.

Задание 9. Просмотр статистики коллекции

Команда:

```
db.magical_creatures.stats()
```

Лог (результат):

```
learn> db.magical_creatures.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata={formatVersion:1},assert={commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=off},block_allocation=best,block_compressor=snappy,cache_resident=false,checksum=on,colgroups=,collators=,columns=,dictionary=,encryption={keyid=,name=},exclusive=false,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_cache_size=false,immutable=false,import={compare_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,panic_corrupt=true,repair=false},internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log={enabled=true},lsm={auto_throttle=true,bloom=true,bloom_bit_count=16,bloom_count=8,bloom_hash_count=8,bloom_oldest=false,chunk_count_limit=0,chunk_max=5GB,chunk_size=10MB,merge_custom={prefix=,start_generation=0,suffix=},merge_max=15,merge_min=0,memory_page_image_max=0,memory_page_max=10M,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_get=0,tiered_storage={auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0},type=file,value_format=u,verbose=},write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection=7-10526000507830488317',
    LSM: {
      'bloom filter false positives': 0,
      'bloom filter hits': 0,
      'bloom filter misses': 0,
      'bloom filter pages evicted from cache': 0,
      'bloom filter pages read into cache': 0,
      'bloom filters in the LSM tree': 0,
      'chunks in the LSM tree': 0,
      'highest merge generation in the LSM tree': 0,
      'queries that could have benefited from a Bloom filter that did not exist': 0,
      'sleep for LSM checkpoint throttle': 0,
      'sleep for LSM merge throttle': 0,
      'total size of bloom filters': 0
    },
    autocommit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
      'total modified incremental blocks without compressed data': 0
    },
    'block-manager': {
      'allocations requiring file extension': 4,
      'blocks allocated': 4,
      'blocks freed': 0,
      'checkpoint size': 4096,
      'file allocation unit size': 4096,
      'file bytes available for reuse': 0,
      'file magic number': 120897,
      'file major version number': 1,
      'file size in bytes': 20480,
      'minor version number': 0
    },
    btree: {
      'btree checkpoint generation': 17,
      'btree clean tree checkpoint expiration time': Long('9223372036854775807'),
      'btree compact pages reviewed': 0,
      'btree compact pages rewritten': 0,
      'btree compact pages skipped': 0,
      'btree expected number of compact bytes rewritten': 0,
      'btree expected number of compact pages rewritten': 0,
      'btree number of pages reconciled during checkpoint': 2,
      'btree skipped by compaction as process would not reduce size': 0,

```

```

'pages written including an aggregated newest stop durable timestamp ': 0,
'pages written including an aggregated newest stop timestamp ': 0,
'pages written including an aggregated newest stop transaction ID': 0,
'pages written including an aggregated newest transaction ID ': 0,
'pages written including an aggregated oldest start timestamp ': 0,
'pages written including an aggregated prepare': 0,
'pages written including at least one prepare': 0,
'pages written including at least one start durable timestamp': 0,
'pages written including at least one start timestamp': 0,
'pages written including at least one start transaction ID': 0,
'pages written including at least one stop durable timestamp': 0,
'pages written including at least one stop timestamp': 0,
'pages written including at least one stop transaction ID': 0,
'records written including a prepare': 0,
'records written including a start durable timestamp': 0,
'records written including a start timestamp': 0,
'records written including a start transaction ID': 0,
'records written including a stop durable timestamp': 0,
'records written including a stop timestamp': 0,
'records written including a stop transaction ID': 0
},
session: { 'object compaction': 0 },
transaction: {
'a reader raced with a prepared transaction commit and skipped an update or updates': 0,
'number of times overflow removed value is read': 0,
'race to read prepared update retry': 0,
'rollback to stable history store keys that would have been swept in non-dryrun mode': 0,
'rollback to stable history store records with stop timestamps older than newer records': 0,
'rollback to stable inconsistent checkpoint': 0,
'rollback to stable keys removed': 0,
'rollback to stable keys restored': 0,
'rollback to stable keys that would have been removed in non-dryrun mode': 0,
'rollback to stable keys that would have been restored in non-dryrun mode': 0,
'rollback to stable restored tombstones from history store': 0,
'rollback to stable restored updates from history store': 0,
'rollback to stable skipping delete rle': 0,
'rollback to stable skipping stable rle': 0,
'rollback to stable sweeping history store keys': 0,
'rollback to stable tombstones from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
'rollback to stable updates removed from history store': 0,
'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
'update conflicts': 0
}
},
sharded: false,
size: 65,
count: 1,
numOrphanDocs: 0,
storageSize: 20480,
totalIndexSize: 20480,
totalSize: 40960,
indexSizes: { _id: 20480 },
avgObjSize: 65,
ns: 'learn.magical_creatures',
nindexes: 1,
scaleFactor: 1
}
learn>

```

Вывод: Коллекция содержит 1 документ, средний размер 65 байт, имеет 1 индекс (по `_id`).

Задание 10. Удаление коллекции

Команда:

```
db.magical_creatures.drop()
```

Лог (результат):

```
[learn> db.magical_creatures.drop()
true
```

Проверка:

```
[learn> show collections
```

```
learn>
```

Вывод: Коллекция успешно удалена.

Задание 11. Удаление БД learn

Команда:

```
db.dropDatabase()
```

Лог (результат):

```
[learn> db.dropDatabase()  
  { ok: 1, dropped: 'learn' }  
learn> █
```

Проверка:

```
[learn> show dbs  
admin    40.00 KiB  
config   92.00 KiB  
local    40.00 KiB  
learn> █
```

Вывод: База данных learn успешно удалена.

4 Промежуточные выводы по ЛР 6.1

В ходе выполнения лабораторной работы 6.1 были получены следующие навыки:

- Установка MongoDB на macOS с использованием Homebrew
- Запуск клиента mongosh и подключение к серверу
- Выполнение базовых команд: `db.help()`, `db.stats()`, `show dbs`, `show collections`
- Создание и переключение между базами данных с помощью `use`
- Создание коллекций явно (`createCollection`) и неявно (при вставке документа)
- Вставка документов с помощью `insertOne()`
- Переименование коллекций с помощью `renameCollection()`
- Просмотр статистики коллекции с помощью `stats()`
- Удаление коллекций и баз данных с помощью `drop()` и `dropDatabase()`

5 Выполнение работы ЛР 6.02

Практическое задание 2.1.1

Команды:

use learn

```
db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires:
182},
  {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f',
vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires:
40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires:
33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires:
54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'},
  {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires:
165}
])
```

db.unicorns.find()

Лог (результат):

```

learn> use learn
already on db learn
learn> db.unicorns.insertMany([
|   {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
|   {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
|   {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
|   {name: 'Roodoodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
|   {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
|   {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
|   {name: 'Kennny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
|   {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
|   {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
|   {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
|   {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'},
|   {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
| ])
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a0ee6bc88f1e93cf751961f'),
|     '1': ObjectId('6a0ee6bc88f1e93cf7519620'),
|     '2': ObjectId('6a0ee6bc88f1e93cf7519621'),
|     '3': ObjectId('6a0ee6bc88f1e93cf7519622'),
|     '4': ObjectId('6a0ee6bc88f1e93cf7519623'),
|     '5': ObjectId('6a0ee6bc88f1e93cf7519624'),
|     '6': ObjectId('6a0ee6bc88f1e93cf7519625'),
|     '7': ObjectId('6a0ee6bc88f1e93cf7519626'),
|     '8': ObjectId('6a0ee6bc88f1e93cf7519627'),
|     '9': ObjectId('6a0ee6bc88f1e93cf7519628'),
|     '10': ObjectId('6a0ee6bc88f1e93cf7519629'),
|     '11': ObjectId('6a0ee6bc88f1e93cf751962a')
|   }
| }
learn> db.unicorns.find()
[
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf751961f'),
|   name: 'Horny',
|   loves: [ 'carrot', 'papaya' ],
|   weight: 600,
|   gender: 'm',
|   vampires: 63
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519620'),
|   name: 'Aurora',
|   loves: [ 'carrot', 'grape' ],
|   weight: 450,
|   gender: 'f',
|   vampires: 43
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519621'),
|   name: 'Unicrom',
|   loves: [ 'energon', 'redbull' ],
|   weight: 984,
|   gender: 'm',
|   vampires: 80
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519624'),
|   name: 'Ayna',
|   loves: [ 'strawberry', 'lemon' ],
|   weight: 733,
|   gender: 'f',
|   vampires: 40
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519625'),
|   name: 'Kennny',
|   loves: [ 'grape', 'lemon' ],
|   weight: 690,
|   gender: 'm',
|   vampires: 39
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519626'),
|   name: 'Raleigh',
|   loves: [ 'apple', 'sugar' ],
|   weight: 421,
|   gender: 'm',
|   vampires: 2
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519627'),
|   name: 'Leia',
|   loves: [ 'apple', 'watermelon' ],
|   weight: 601,
|   gender: 'f',
|   vampires: 33
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519628'),
|   name: 'Pilot',
|   loves: [ 'apple', 'watermelon' ],
|   weight: 650,
|   gender: 'm',
|   vampires: 54
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf7519629'),
|   name: 'Nimue',
|   loves: [ 'grape', 'carrot' ],
|   weight: 540,
|   gender: 'f'
| },
| {
|   _id: ObjectId('6a0ee6bc88f1e93cf751962a'),
|   name: 'Dunx',
|   loves: [ 'grape', 'watermelon' ],
|   weight: 704,
|   gender: 'm',
|   vampires: 165
| }
]

```

Вывод: Все 12 документов успешно вставлены в коллекцию unicorns.

Практическое задание 2.2.1

Команды:

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

Лог (результат):

Список самцов

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf751962a'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf751961f'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519625'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519628'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519626'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519622'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519621'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

Список самок (первые 3)

```
[learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519620'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519624'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519627'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

Самки, любящие carrot (через findOne)

```
[learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0ee6bc88f1e93cf7519620'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> █
```

Самки, любящие carrot (через limit)

```
[learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519620'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

Вывод: Метод findOne() сразу возвращает документ, а find().limit(1) возвращает курсор.

Практическое задание 2.2.2

Команда:

```
db.unicorns.find({gender: 'm'}, {loves: 0, _id: 0})
```

Лог (результат):

```
[learn> db.unicorns.find({gender: 'm'}, {loves: 0, _id: 0})
[
  { name: 'Horny', weight: 600, gender: 'm', vampires: 63 },
  { name: 'Unicrom', weight: 984, gender: 'm', vampires: 182 },
  { name: 'Roooooodles', weight: 575, gender: 'm', vampires: 99 },
  { name: 'Kenny', weight: 690, gender: 'm', vampires: 39 },
  { name: 'Raleigh', weight: 421, gender: 'm', vampires: 2 },
  { name: 'Pilot', weight: 650, gender: 'm', vampires: 54 },
  { name: 'Dunx', weight: 704, gender: 'm', vampires: 165 }
]
learn>
```

Вывод: Второй параметр find() позволяет указать, какие поля включить (1) или исключить (0).

Практическое задание 2.2.3

Команда:

```
db.unicorns.find().sort({$natural: -1})
```

Лог (результат):

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a8e0d0c88f1e93cf751962a'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519629'),
    name: 'Unicrom',
    loves: [ 'grape', 'carrot' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519628'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519627'),
    name: 'Kenny',
    loves: [ 'apple', 'watermelon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519626'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519625'),
    name: 'Horny',
    loves: [ 'grape', 'lemon' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519624'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 735,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a8e0d0c88f1e93cf7519623'),
    name: 'Roooooodles',
    loves: [ 'apple', 'watermelon' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  }
]
```

```

{
  _id: ObjectId('6a0ee6bc88f1e93cf7519623'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a0ee6bc88f1e93cf7519622'),
  name: 'Roooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
{
  _id: ObjectId('6a0ee6bc88f1e93cf7519621'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a0ee6bc88f1e93cf7519620'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a0ee6bc88f1e93cf751961f'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
]

```

Вывод: \$natural: -1 задает сортировку в обратном порядке хранения на диске.

Практическое задание 2.1.4

Команда:

```
db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
```

Лог (результат):

```

[learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>

```

Вывод: \$slice: 1 возвращает первый элемент массива loves.

Практическое задание 2.3.1

Команда:

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

Лог (результат):

```
[learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> █
```

Вывод: \$gte ($\geq$) и \$lte ($\leq$) задают диапазон значений.

Практическое задание 2.3.2

Команда:

```
db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
```

Лог (результат):

```
[learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Вывод: \$all требует наличия всех указанных элементов в массиве.

Практическое задание 2.3.3

Команда:

```
db.unicorns.find({vampires: {$exists: false}})
```

Лог (результат):

```
[learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519629'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Вывод: \$exists: false находит документы, у которых поле отсутствует.

Практическое задание 2.3.4

Команда:

```
db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
```

Лог (результат):

```
[learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Вывод: Комбинация \$slice, проекции и сортировки дает структурированный результат.

Практическое задание 3.1.1

Команды:

```
use towns_db
```

```
db.towns.insertMany([
  {name: "Punxsutawney", population: 6200, last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"], mayor: {name: "Jim Wehrle"}},
  {name: "New York", population: 22200000, last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"], mayor: {name: "Michael Bloomberg",
    party: "I"}},
  {name: "Portland", population: 528000, last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"], mayor: {name: "Sam Adams", party: "D"}}
])
```

```
db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
```

```
db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
```

Лог (результат):

```
[learn> use towns_db
switched to db towns_db
towns_db> db.towns.insertMany([
  {name: "Punxsutawney", population: 6200, last_sensus: ISODate("2008-01-31"), famous_for: ["phil the groundhog"], mayor: {name: "Jim Wehrle"}},
  {name: "New York", population: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: ["statue of liberty", "food"], mayor: {name: "Michael Bloomberg", party
: "I"}},
  {name: "Portland", population: 528000, last_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor: {name: "Sam Adams", party: "D"}}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0eec4888f1e93cf751962b'),
    '1': ObjectId('6a0eec4888f1e93cf751962c'),
    '2': ObjectId('6a0eec4888f1e93cf751962d')
  }
}
```

Независимые мэры

```
[towns_db> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
towns_db> █
```

Беспартийные мэры

```
[towns_db> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
towns_db> █
```

Вывод: Для доступа к вложенным полям используется точка (mayor.party)

Практическое задание 3.1.2

Команды:

```
use learn
```

```
function showMales() {  
  db.unicorns.find({gender: 'm'}).forEach(function(doc) {  
    print(doc.name)  
  })  
}  
showMales()
```

```
var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)  
cursor.forEach(function(obj) { print(obj.name); })
```

Лог (результат):

Функция

```
[towns_db> use learn  
switched to db learn  
learn> function showMales() {  
|   db.unicorns.find({gender: 'm'}).forEach(function(doc) {  
|     print(doc.name)  
|   })  
| }  
| showMales()  
[|  
Horny  
Unicrom  
Rooooooodles  
Kenny  
Raleigh  
Pilot  
Dunx  
  
learn> █
```

Курсор

```
learn> var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)  
| cursor.forEach(function(obj) { print(obj.name); })  
[|  
Dunx  
Horny  
  
learn> █
```

Вывод: Курсоры позволяют обрабатывать документы по одному. Методы `limit()`, `sort()`, `skip()` объединяются в цепочки.

Практическое задание 3.2

Команды:

```
db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
db.unicorns.distinct("loves")
db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]])
```

Лог (результат):

Количество самок 500-600 кг

```
[learn> db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
2
_
```

Уникальные предпочтения

```
[learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> █
```

Агрегация по полу

```
[learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn> █
```

Вывод: countDocuments() — для подсчета, distinct() — для уникальных значений, aggregate() с \$group — для группировки.

Практическое задание 3.3.1

Команда:

```
db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
```

Лог (результат):

```
[learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a0eee5388f1e93cf751962e')
}
learn> █
```

Проверка:

```
learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a0eee5388f1e93cf751962e'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Вывод: Документ с именем Barney успешно добавлен в коллекцию. MongoDB автоматически сгенерировала уникальный идентификатор `_id`.

Практическое задание 3.3.2

Команда:

```
db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
```

Лог (результат):

```
[learn> db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> █
```

Проверка:

```
[learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519624'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn> █
```

Вывод: Оператор `$set` успешно обновил поля `weight` (с 733 на 800) и `vampires` (с 40 на 51). Остальные поля остались без изменений.

Практическое задание 3.3.3

Команда:

```
db.unicorns.updateOne({name: 'Raleigh'}, {$push: {loves: 'redbull'}})
```

Лог (результат):

```
[learn> db.unicorns.updateOne({name: 'Raleigh'}, {$push: {loves: 'redbull'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> █
```

Проверка:

```
[learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519626'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> █
```

Вывод: Оператор \$push добавил новый элемент 'redbull' в массив loves. Ранее в массиве были 'apple' и 'sugar'.

Практическое задание 3.3.4

Команда:

```
db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
```

Лог (результат):

```
[learn> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> █
```

Проверка:

```
[learn> db.unicorns.find({gender: 'm'}, {name: 1, vampires: 1, _id: 0})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barney', vampires: 5 }
]
learn> █
```

Вывод:

updateMany() обновил всех самцов (8 документов)

\$inc увеличил значение поля vampires на 5 для каждого

У Barney не было поля vampires — оператор \$inc создал его со значением 5 (0 + 5)

Практическое задание 3.3.5

Команды:

```
use towns_db
```

```
db.towns.updateOne({name: "Portland"}, {$unset: {party: ""}})
```

```
db.towns.find({name: "Portland"})
```

Лог (результат):

```
[learn> use towns_db
switched to db towns_db
[towns_db> db.towns.updateOne({name: "Portland"}, {$unset: {party: ""}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
[towns_db> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a0eec4888f1e93cf751962d'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
towns_db> █
```

Вывод: Оператор \$unset удалил поле party из вложенного документа mayor.

Раньше было mayor: { name: 'Sam Adams', party: 'D' }, теперь party отсутствует.

Практическое задание 3.3.6

Команды:

```
use learn
db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
db.unicorns.find({name: 'Pilot'})
```

Лог (результат):

```
[towns_db> use learn
switched to db learn
[learn> db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519628'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> █
```

Вывод: \$push добавил 'chocolate' в конец массива loves. Ранее в массиве были 'apple' и 'watermelon'.

Практическое задание 3.3.7

Команды:

```
db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
db.unicorns.find({name: 'Aurora'})
```

Лог (результат):

```
[learn> db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a0ee6bc88f1e93cf7519620'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

Вывод:

\$addToSet добавляет элементы только если их еще нет в массиве

\$each позволяет добавить сразу несколько элементов

В массив добавлены 'sugar' и 'lemon', при этом 'carrot' и 'grape' остались

Практическое задание 3.4.1

Команды:

use towns_db

```
db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {name: "Jim Wehrle"}
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {name: "Michael Bloomberg", party: "I"}
  },
  {

```



```

    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {name: "Sam Adams", party: "D"}
  }
])

```

```
db.towns.deleteMany({"mayor.party": {$exists: false}})
```

```
db.towns.find()
```

```
db.towns.deleteMany({})
```

```
show collections
```

Лог (результат):

```

[learn> use towns_db
switched to db towns_db
towns_db> db.towns.insertMany([
|   {
|     name: "Punxsutawney",
|     population: 6200,
|     last_sensus: ISODate("2008-01-31"),
|     famous_for: ["phil the groundhog"],
|     mayor: {name: "Jim Wehrle"}
|   },
|   {
|     name: "New York",
|     population: 22200000,
|     last_sensus: ISODate("2009-07-31"),
|     famous_for: ["statue of liberty", "food"],
|     mayor: {name: "Michael Bloomberg", party: "I"}
|   },
|   {
|     name: "Portland",
|     population: 528000,
|     last_sensus: ISODate("2009-07-20"),
|     famous_for: ["beer", "food"],
|     mayor: {name: "Sam Adams", party: "D"}
|   }
| ])
[]
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0ef35e88f1e93cf751962f'),
    '1': ObjectId('6a0ef35e88f1e93cf7519630'),
    '2': ObjectId('6a0ef35e88f1e93cf7519631')
  }
}
[towns_db> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 2 }

```



```
[towns_db> db.towns.find()
[
  {
    _id: ObjectId('6a0eec4888f1e93cf751962c'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0eec4888f1e93cf751962d'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  },
  {
    _id: ObjectId('6a0ef35e88f1e93cf7519630'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0ef35e88f1e93cf7519631'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

```
[towns_db> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
[towns_db> show collections
towns
towns_db>
```

Вывод:

`deleteMany()` с условием удаляет все документы, соответствующие критерию

`deleteMany({})` удаляет все документы из коллекции, но сама коллекция остается

Коллекция `towns` все еще существует, даже после удаления всех документов

Практическое задание 4.1.1

Команды:

`use learn`

`db.habitats.insertMany([`

```

    { _id: "forest", name: "Дремучий лес", description: "Густой лес с магическими источниками" },
    { _id: "mountains", name: "Горные вершины", description: "Высокие горы с кристальными пещерами" },
    { _id: "valley", name: "Изумрудная долина", description: "Цветущая долина с радугами" }
  ])

```

```

db.unicorns.updateOne({name: "Horny"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
db.unicorns.updateOne({name: "Aurora"}, {$set: {habitat: {$ref: "habitats", $id: "valley"}}})
db.unicorns.updateOne({name: "Unicrom"}, {$set: {habitat: {$ref: "habitats", $id: "mountains"}}})

```

```

db.unicorns.find({}, {name: 1, habitat: 1, _id: 0})

```

```

var horny = db.unicorns.findOne({name: "Horny"})
db.habitats.findOne({_id: horny.habitat.$id})

```

Лог (результат):

```

[learn> use learn
already on db learn
learn> db.habitats.insertMany([
|   { _id: "forest", name: "Дремучий лес", description: "Густой лес с магическими источниками" },
|   { _id: "mountains", name: "Горные вершины", description: "Высокие горы с кристальными пещерами" },
|   { _id: "valley", name: "Изумрудная долина", description: "Цветущая долина с радугами" }
| ])
[]
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains', '2': 'valley' }
}
learn> db.unicorns.updateOne({name: "Horny"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
| db.unicorns.updateOne({name: "Aurora"}, {$set: {habitat: {$ref: "habitats", $id: "valley"}}})
| db.unicorns.updateOne({name: "Unicrom"}, {$set: {habitat: {$ref: "habitats", $id: "mountains"}}})
[]
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({}, {name: 1, habitat: 1, _id: 0})
[
  { name: 'Horny', habitat: DBRef('habitats', 'forest') },
  { name: 'Aurora', habitat: DBRef('habitats', 'valley') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountains') },
  { name: 'Rooooooodles' },
  { name: 'Solnara' },
  { name: 'Ayna' },
  { name: 'Kenny' },
  { name: 'Raleigh' },
  { name: 'Leia' },
  { name: 'Pilot' },
  { name: 'Nimue' },
  { name: 'Dunx' },
  { name: 'Barney' }
]

```

```
learn> var horny = db.unicorns.findOne({name: "Horny"})
[| horny.habitat
DBRef('habitats', 'forest')
learn> █
```

Вывод:

DBRef — это стандартный формат ссылок в MongoDB: {\$ref: "коллекция", \$id: "значение"}

Ссылки не поддерживают автоматическую целостность (нет аналога FOREIGN KEY)

Для получения связанных данных нужно выполнять отдельный запрос

Практическое задание 4.2.1

Команда:

```
use learn
db.unicorns.createIndex({name: 1}, {unique: true})
```

Лог (результат):

```
[towns_db> use learn
switched to db learn
[learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
learn> █
```

Вывод: Индекс name_1 успешно создан

Практическое задание 4.3.1

Команды:

```
db.unicorns.getIndexes()
db.unicorns.dropIndex("name_1")
db.unicorns.dropIndex("_id_")
```

Лог (результат):

```
[learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_',
    { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
[learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
[learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> █
```

Вывод:

getIndexes() показывает все индексы коллекции

Индекс по полю `_id` создается автоматически

Индекс `_id` нельзя удалить — он необходим для уникальной идентификации документов

Практическое задание 4.4.1

Команды:

```
use learn
for(i = 0; i < 100000; i++){ db.numbers.insertOne({value: i}) }
db.numbers.find().sort({$natural: -1}).limit(4)
db.numbers.find().sort({$natural: -1}).limit(4).explain("executionStats")
db.numbers.createIndex({value: 1})
db.numbers.getIndexes()
db.numbers.find().sort({value: -1}).limit(4)
db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
```

Лог (результат):

Создание коллекции numbers (100 000 документов)

```
[learn> for(i = 0; i < 100000; i++){ db.numbers.insertOne({value: i}) }
{
  acknowledged: true,
  insertedId: ObjectId('6a0ef5ac88f1e93cf7531cd1')
}
```

Выбор последних четырех документов (без индекса)

```
[learn> db.numbers.find().sort({$natural: -1}).limit(4)
[
  { _id: ObjectId('6a0ef5ac88f1e93cf7531cd1'), value: 99999 },
  { _id: ObjectId('6a0ef5ac88f1e93cf7531cd0'), value: 99998 },
  { _id: ObjectId('6a0ef5ac88f1e93cf7531ccf'), value: 99997 },
  { _id: ObjectId('6a0ef5ac88f1e93cf7531cce'), value: 99996 }
]
```

Анализ плана запроса без индекса

```

[learn> db.numbers.find().sort({$natural: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: '8F2383EE',
    planCacheShapeHash: '0F2383EE',
    planCacheKey: '7DF350EE',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: { stage: 'COLLSCAN', direction: 'backward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 4,
      inputStage: {
        stage: 'COLLSCAN',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 4,
        advanced: 4,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 0,
        direction: 'backward',
        docsExamined: 4
      }
    }
  }
},
  queryShapeHash: '71C04F100DB2038344A2B08A520E31838D2BDF488D60563B2C7886E847B22DE4',
  command: {
    find: 'numbers',
    filter: {},
    sort: { '$natural': -1 },
    limit: 4,
    '$db': 'learn'
  },
  serverInfo: {
    host: 'Polinas-MacBook-Pro.local',
    port: 27017,
    version: '8.0.23',
    gitVersion: 'ccf0d81588377542f00eeecf1b1ffbc095b1eeff'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
[learn> ]

```

Создание индекса по полю value

```

[learn> db.numbers.createIndex({value: 1})
value_1
learn> ]

```


Получение информации об индексах

```
[learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>
```

Выбор последних четырех документов (С индексом)

```
[learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a0ef5ac88f1e93cf7531cd1'), value: 99999 },
  { _id: ObjectId('6a0ef5ac88f1e93cf7531cd0'), value: 99998 },
  { _id: ObjectId('6a0ef5ac88f1e93cf7531ccf'), value: 99997 },
  { _id: ObjectId('6a0ef5ac88f1e93cf7531cce'), value: 99996 }
]
learn>
```

Анализ плана запроса с индексом

```
[learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'BA27D965',
    planCacheShapeHash: 'BA27D965',
    planCacheKey: '7AB92BB1',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExploreReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stages: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 4,
    }
  }
}
```

```

alreadyHasObj: 0,
inputStage: {
  stage: 'IXSCAN',
  nReturned: 4,
  executionTimeMillisEstimate: 0,
  works: 4,
  advanced: 4,
  needTime: 0,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
  isEOF: 0,
  keyPattern: { value: 1 },
  indexName: 'value_1',
  isMultiKey: false,
  multiKeyPaths: { value: [] },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'backward',
  indexBounds: { value: [ '[MaxKey, MinKey]' ] },
  keysExamined: 4,
  seeks: 1,
  dupsTested: 0,
  dupsDropped: 0
}
},
},
queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
command: {
  find: 'numbers',
  filter: {},
  sort: { value: -1 },
  limit: 4,
  '$db': 'learn'
},
serverInfo: {
  host: 'Polinas-MacBook-Pro.local',
  port: 27017,
  version: '8.0.23',
  gitVersion: 'ccf0d81588377542f00eeecf1b1ffbc095b1eeff'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbsRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
-

```

Вывод:

В данном конкретном случае разница во времени выполнения незаметна из-за малого количества возвращаемых документов (LIMIT 4)

Однако важно отметить, что запрос с использованием индекса (IXSCAN) является более правильным с точки зрения оптимизации

На больших коллекциях (миллионы документов) разница в производительности становится критической: COLLSCAN сканирует всю коллекцию, а IXSCAN — только нужные записи

Индексы особенно эффективны при сортировке и фильтрации данных

6 Промежуточные выводы по ЛР 6.2

В ходе выполнения лабораторной работы 6.2 были освоены следующие навыки:

1. CRUD-операции

Create: Вставка документов с помощью `insertOne()` и `insertMany()`

Read: Выборка данных с помощью `find()` и `findOne()`

Update: Обновление документов с помощью `updateOne()` и `updateMany()`

Delete: Удаление документов с помощью `deleteOne()` и `deleteMany()`

2. Операторы запросов

Сравнения: `$gt`, `$lt`, `$gte`, `$lte`

Работа с массивами: `$all`, `$slice`, `$push`, `$addToSet`

Существование поля: `$exists`

Обновления: `$set`, `$inc`, `$unset`

3. Агрегация данных

Подсчет количества документов с помощью `countDocuments()`

Получение уникальных значений с помощью `distinct()`

Группировка данных с помощью `aggregate()` и `$group`

4. Работа с вложенными документами

Доступ к вложенным полям через оператор точка (`.`)

Обновление вложенных полей

5. Курсоры

Создание курсоров для пошаговой обработки результатов

Использование методов `forEach()`, `hasNext()`, `next()`

6. Индексы и оптимизация

Создание индексов с помощью `createIndex()`

Создание уникальных индексов (`unique: true`)

Просмотр индексов с помощью `getIndexes()`

Удаление индексов с помощью `dropIndex()`

Анализ планов запросов с помощью `explain("executionStats")`

7. Ссылки между коллекциями (DBRef)

Создание ссылок формата {\$ref: "коллекция", \$id: "значение"}

Получение связанных данных по ссылке

7 Вывод

В результате выполнения лабораторных работ 6.1 и 6.2 я полностью освоила основы работы с документо-ориентированной СУБД MongoDB. Я научилась устанавливать MongoDB на macOS, выполнять все основные CRUD-операции, создавать индексы для оптимизации запросов, анализировать планы выполнения запросов и работать со ссылками между коллекциями.

MongoDB является мощной альтернативой реляционным СУБД, особенно в проектах, где важны гибкость схемы данных, высокая скорость разработки и горизонтальная масштабируемость. Полученные навыки являются фундаментом для дальнейшего изучения NoSQL-технологий и их применения в реальных проектах.